A beginner friendly intro to functional programming

Johann Schuster
Oct 25, 2017 · 7 min read

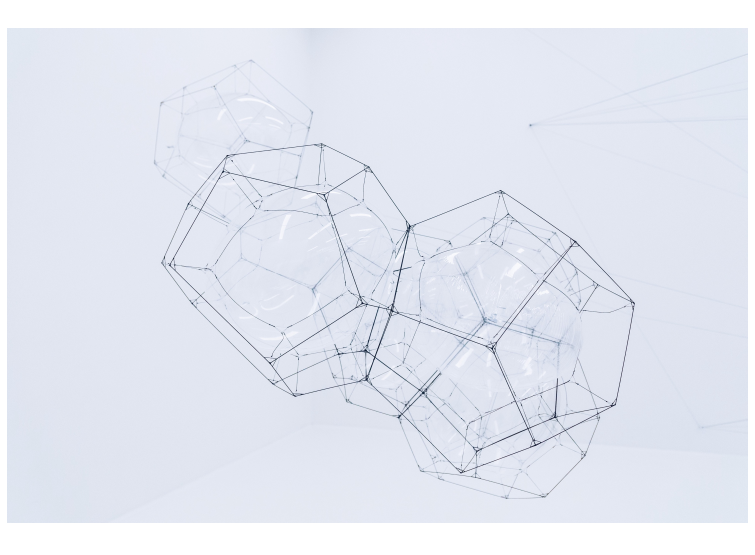


Photo by Sebastien Gabriel on Unsplash

Functional programming is a highly valued approach to writing code, and it's popularity is continuously increasing in commercial software applications. Functional programming is a paradigm of writing code and is eloquently put in the introduction of this Wikipedia article:

"-a style of building the structure and elements of computer programs — that treats computation as the evaluation of mathematical functions and avoids changing-state and mutable data. It is a declarative programming paradigm, which means programming is is done with expressions or declarations instead of statements."

We will get into what this exactly means very soon, but first, let's get the fundamentals out of the way by providing some context. This article serves the purpose of hopefully introducing the fundamental concepts of functional programming to you and by the end of it, we'd have established a basic understanding of why we would choose to apply it into our coding styles, how it relates to JavaScript and some basic patterns that you can begin implementing into your code. Be on the lookout for a second article that will build on this one and dig even deeper into the ideas discussed here.

The roots of functional programming lie in Lambda calculus, which was a system developed in the 1930's to express computation using functions similar to what we will see very soon. For quite a while, functional programming concepts and languages were widely discussed in academia and scholarly contexts and eventually bled into the development of commercial software. What makes a programming language functional or not is its ability to facilitate the functional programming paradigm, languages like these are Ruby, Python, Julia and JavaScript, the latter of which has properties of imperative, object-oriented and functional paradigms.

Let's look at an example of using an imperative, and then a functional approach to completing a single task.

Imperative:

```
const arr = [1, 2, 3, 4, 5, 6 ,7 ,8 ,9]

function getOdds(arr){
   let odds = [ ];

      for(let i = 0; i < arr.length + 1; i++){
            if ( i % 2 !== 0 ){
               odds.push( i )
            };
         };
      return odds
   };

console.log(getOdds(arr))
// logs [1, 3, 5, 7, 9]
```

Functional:

```
function getOdds2(arr){
   return arr.filter(num => num % 2 !== 0)
}

console.log(getOdds2(arr))
// logs [ 1, 3, 5, 7, 9 ]

// this can be even shorter
const getOdds3 = arr => arr.filter(num => num % 2 !== 0)

console.log(getOdds3(arr))
// logs [ 1, 3, 5, 7, 9 ]
```

Aside from the obvious improvements the functional approach makes, like shorter code, saving us a couple of keystrokes, and the fact that it's much easier to understand at a quick glance, it's also elegant, which results in cleaner code, simplifies debugging, testing and maintenance.

Let's break this down a bit, so that we can reach a clear understanding of the differences between the two approaches and define what functional programming is.

In the first approach we are imperatively describing every event that needs to take place in order for us to achieve the task of filtering out the odd numbers from an array. We feed a function an array, inside the function we create an empty array which we will use to store the odd numbers in. We tell it to loop over the array, then we declare a conditional, if the current index is an odd number, push it to the empty one. The for loop does not include the last number in the array, it only includes the numbers leading up to it, so if our last number is odd, it won't be included. Therefore we add +1 when defining the amount of iterations our loop needs to make.

In the second approach, we simply define the outcome we want to see by using a method called filter, and allow the machine to take care of all the steps in between. This is a more declarative approach. We declare what the end result must be, and we the rest is handled for us.

Now we can see what the above mentioned definition strives to express. A way of programming, using declarations and expressions, rather than statements. This definition only scrapes the surface of functional programming, and we will continually build on our definition as our understanding grows.

Some differences between iterative and functional approaches.

- Stylistically, we define a problem and the changes we would like to see, as opposed to explaining it step by step.
- We do not need to manage state in our functions like above where we managed the state of our empty array.
- We dont have to worry as much about the order of execution.
- We use less loops, conditions and use more built in methods, reusable functions and recursion.

By using this approach, we ultimately write cleaner code which as mentioned above, helps us in debugging and maintenance. It keeps things modular, and sections of our code is broken down into smaller pieces that we can test with ease. We can reuse parts of our code using helper functions and lastly, it's a more efficient, mathematical approach to write code.

The Pillars: Our coding toolkit for writing functional code

The following are a few important aspects to writing high quality functional code. Seeing that this is an overview, we won't dive in too much into each, instead we will just define a few key characteristics and one example of each.

Pure Functions:

Pure functions return a value solely based on what was passed into it. The idea behind it is that it will be a pure function if it always returns the same result if the same values are passed into it, and it doesn't modify values outside of ts scope, that makes it independent from any state in the system. Therefore, a pure function can never mutate data, produce no side effects and can be easily reused. An example of a non pure function would be a function that makes an API call or returns an unpredictable result.

Simple Impure function:

```
var tip = 0;

function calculateTip( mealTotal ) {
   tip = 0.15 * mealTotal;
}

calculateTip( 150 )
console.log(tip)
```

Simple Pure function:

```
function isPure(x,y) {
   return x * y
}

console.log(isPure(3,5));
```

The pure function will return the exact result every time, and it doesn't mutate any data outside of it. The important thing to pay attention to is that a pure function must stay away from mutating data whatsoever, so be careful when selecting which methods to use inside your pure functions. For instance, if you wish to merge two arrays inside your function, as one would typically do in a React reducer, avoid using the Array.prototype.push() method. Instead you'd want to use Array.prototype.concat( ), which will preserve the state of the old arrays and return a new one for your use.

Higher order functions

In Javascript, functions are treated as objects, therefore we can pass functions around as we would any other value. A higher order function is simply a function that operates on other functions. They can take in a function as input, or return one as output. We will see a simple example in the following demonstration.

Anonymous Functions

Anonymous functions are very helpful when we need to define ad hoc logic as we need. As the name implies, an anonymous function in nameless, and mostly found as an argument to a function as a replacement for a named function, a function assigned to a variable, or returned as a function in a higher order function. It's roots lie heavily in Lambda calculus and is very important to all functional programming languages.

Anonymous function assigned to a variable. Easy to pass around and invoke when needed.

```
const myVar = function(){console.log('Anonymous function here!')}

myVar()
```

Anonymous function as argument

```
setInterval(function(){console.log(new Date().getTime())}, 1000);
```

Anonymous functions within a higher order function

```
function mealCall(meal){
   return function(message){
      return console.log(message + " " + meal + '!!')
      }
   }

const announceDinner = mealCall('dinner')
const announceLunch = mealCall('breakfast')

announceDinner('hey!, come and get your')
announceLunch('Rise and shine! time for')
```

Recursion

Recursion is a well known functional programming technique. A recursive function is simply a function that calls itself, and therefore acts as a loop executing the same code multiple times. It comes with one caveat though, one must be careful to avoid infinite recursions. Therefore a base case is needed, to tell it when to stop. Recursive functions or algorithms can both be implemented using recursion and loops, but it's better to use recursion. Below is the well know algorithm for factorizing a number both using recursion and a loop.

Recursive function

```
function factorialize(num){
   if (num === 0 || num === 1)
{return 1;}
      return (num * factorialize(num — 1));
   }

var result = factorialize(14);

console.log(result);
```

Recursion using a loop

```
function factorialize(num) {
   if (num === 0 || num === 1){
      return 1;
   }
   for (var i = num-1; i >= 1; i—- ) {
      num *= i;
   }
   return num;
}
console.log(factorialize(6));
```

We have covered a few of the essential ingredients to writing functional code. It is not possible to always write functional code in an application, but the benefits are great when used as much as possible. Frameworks like Angular and React are heavily influenced by functional

Get started

benefit to start thinking in the FP way.

Resources for learning more? Here are some great articles and resources for further learning.

- Why Learn Functional Programming in JavaScript? by Eric Elliot
- Eloquent JavaScript by Marijn Haverbeke
- So you want to be a Functional Programmer by Charles Scalfani

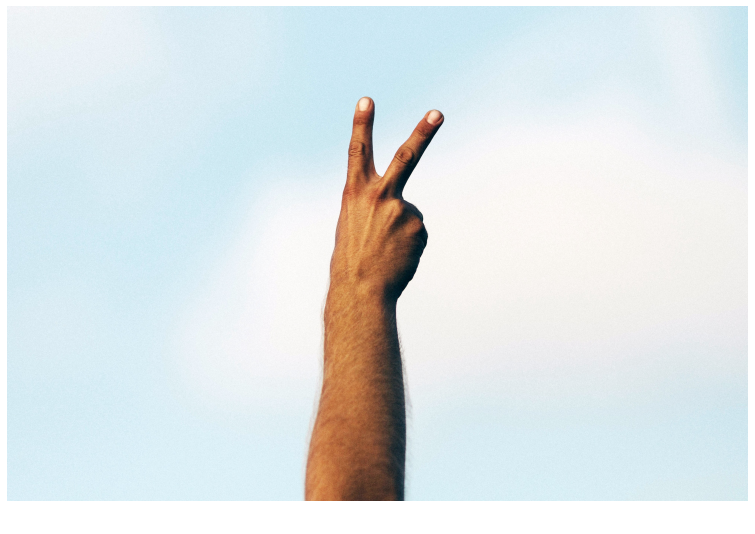


Photo by Patrick Fore on Unsplash

I really hope this was helpful and in the next article we'll dive a little deeper and unpack even more interesting and helpful tools like JavaScript methods, partial application, currying, argument binding and more.

Feel free to clap, share, critique or share any thoughts on the topic!

Functional Programming
JavaScript
Javascript Tips
Web Development
Software Development

956 claps

WRITTEN BY

Johann Schuster

I'm a web developer in LA into startups and solving pressing problems using technology.

Follow

codeburst

Bursts of code to power through your day. Web Development articles, tutorials, and news.

Follow

See responses (1)

More From Medium

More from codeburst

Your React App Is a Mess Before You Even Know It

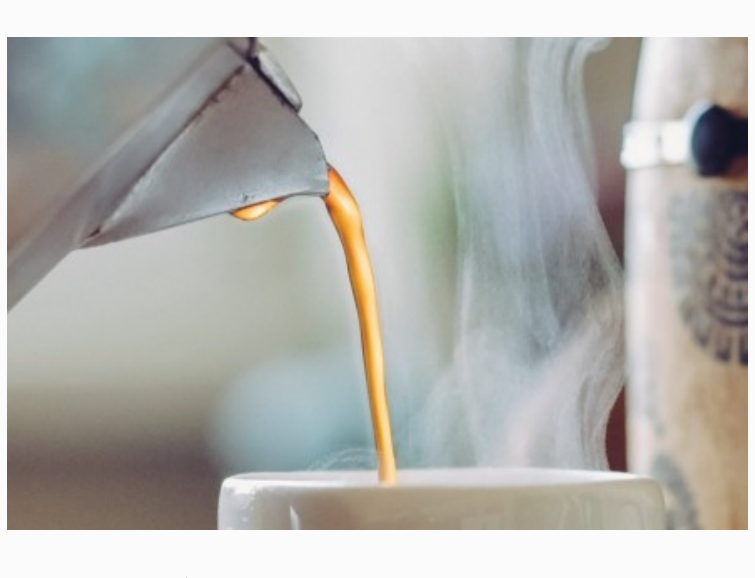
Frank Zickert in codeburst
Sep 17 · 6 min read ★ 336

More from codeburst

How to handle deeply-nested nullable fields in JavaScript and TypeScript

Nadeesha Ca... codeburst
Sep 15 · 4 min read ★ 294

More from codeburst



Dependency injection container with hot-reloading in Node.js

Michał Męci... codeburst
Sep 12 · 6 min read ★ 297